Solution: Minimum Subset Sum Difference

Problem Statement

Given a set of positive numbers, partition the set into two subsets with a minimum difference between their subset sums.

Example 1:

```
Input: {1, 2, 3, 9}
Output: 3
Explanation: We can partition the given set into two subsets where the minimum absolute difference
between the sum of numbers is '3'. Following are the two subsets: {1, 2, 3} & {9}.
```

Example 2:

```
Input: {1, 2, 7, 1, 5}
Output: 0
Explanation: We can partition the given set into two subsets where the minimum absolute difference
between the sum of numbers is '0'. Following are the two subsets: {1, 2, 5} & {7, 1}.
```

Example 3:

```
Input: {1, 3, 100, 4}
Output: 92
Explanation: We can partition the given set into two subsets where the minimum absolute difference
between the sum of numbers is '92'. Here are the two subsets: {1, 3, 4} & {100}.
```

Basic Solution

This problem follows the **0/1 Knapsack pattern** and can be converted into a `"Subset Sum"` problem.

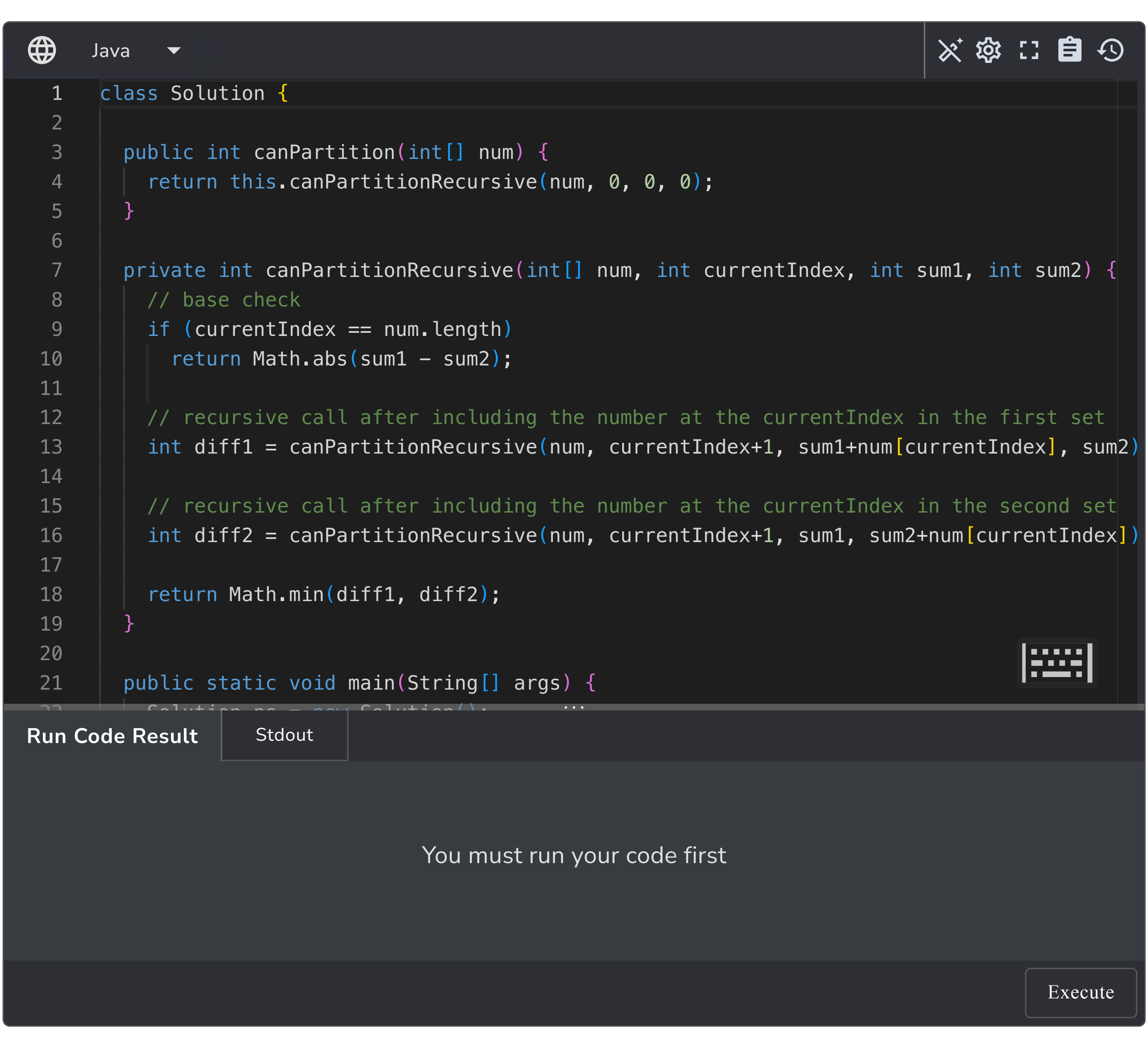
Let's assume S1 and S2 are the two desired subsets. A basic brute-force solution could be to try adding each element either in S1 or S2, to find the combination that gives the minimum sum difference between the two sets.

So our brute-force algorithm will look like:

```
for each number 'i'
  add number 'i' to S1 and recursively process the remaining numbers
  add number 'i' to S2 and recursively process the remaining numbers
return the minimum absolute difference of the above two sets
```

Code

Here is the code for the brute-force solution:



```java
class Solution {

  public int canPartition(int[] num) {
    return this.canPartitionRecursive(num, 0, 0, 0);
  }

  private int canPartitionRecursive(int[] num, int currentIndex, int sum1, int sum2) {
    // base check
    if (currentIndex == num.length)
      return Math.abs(sum1 - sum2);

    // recursive call after including the number at the currentIndex in the first set
    int diff1 = canPartitionRecursive(num, currentIndex+1, sum1+num[currentIndex], sum2)

    // recursive call after including the number at the currentIndex in the second set
    int diff2 = canPartitionRecursive(num, currentIndex+1, sum1, sum2+num[currentIndex])

    return Math.min(diff1, diff2);
  }

  public static void main(String[] args) {
```

You must run your code first

The time complexity of the above algorithm is exponential $O(2^n)$, where ‘n’ represents the total number. The space complexity is $O(n)$ which is used to store the recursion stack.

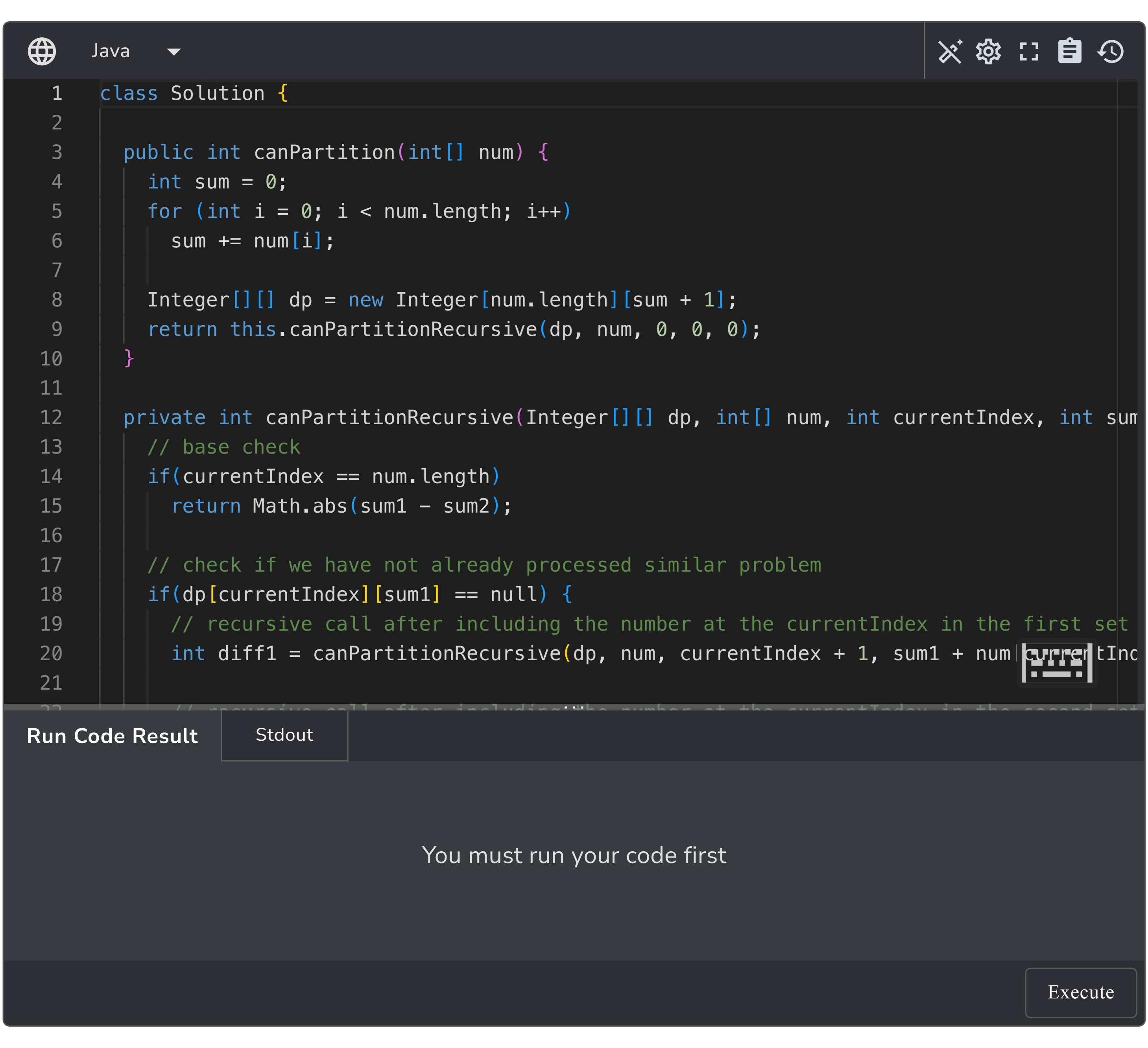
Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems.

We will be using a two-dimensional array to store the results of the solved sub-problems. We can uniquely identify a sub-problem from 'currentIndex' and 'Sum1'; as 'Sum2' will always be the sum of the remaining numbers.

Code

Here is the code:



```java
class Solution {

  public int canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    Integer[][] dp = new Integer[num.length][sum + 1];
    return this.canPartitionRecursive(dp, num, 0, 0, 0);
  }

  private int canPartitionRecursive(Integer[][] dp, int[] num, int currentIndex, int sum
    // base check
    if(currentIndex == num.length)
      return Math.abs(sum1 - sum2);

    // check if we have not already processed similar problem
    if(dp[currentIndex][sum1] == null) {
      // recursive call after including the number at the currentIndex in the first set
      int diff1 = canPartitionRecursive(dp, num, currentIndex + 1, sum1 + num[currentInd

```

You must run your code first

Bottom-up Dynamic Programming

Let's assume 'S' represents the total sum of all the numbers. So what we are trying to achieve in this problem is to find a subset whose sum is as close to 'S/2' as possible, because if we can partition the given set into two subsets of an equal sum, we get the minimum difference i.e. zero. This transforms our problem to `"Subset Sum"`, where we try to find a subset whose sum is equal to a given number-- 'S/2' in our case. If we can't find such a subset, then we will take the subset which has the sum closest to 'S/2'. This is easily possible, as we will be calculating all possible sums with every subset.

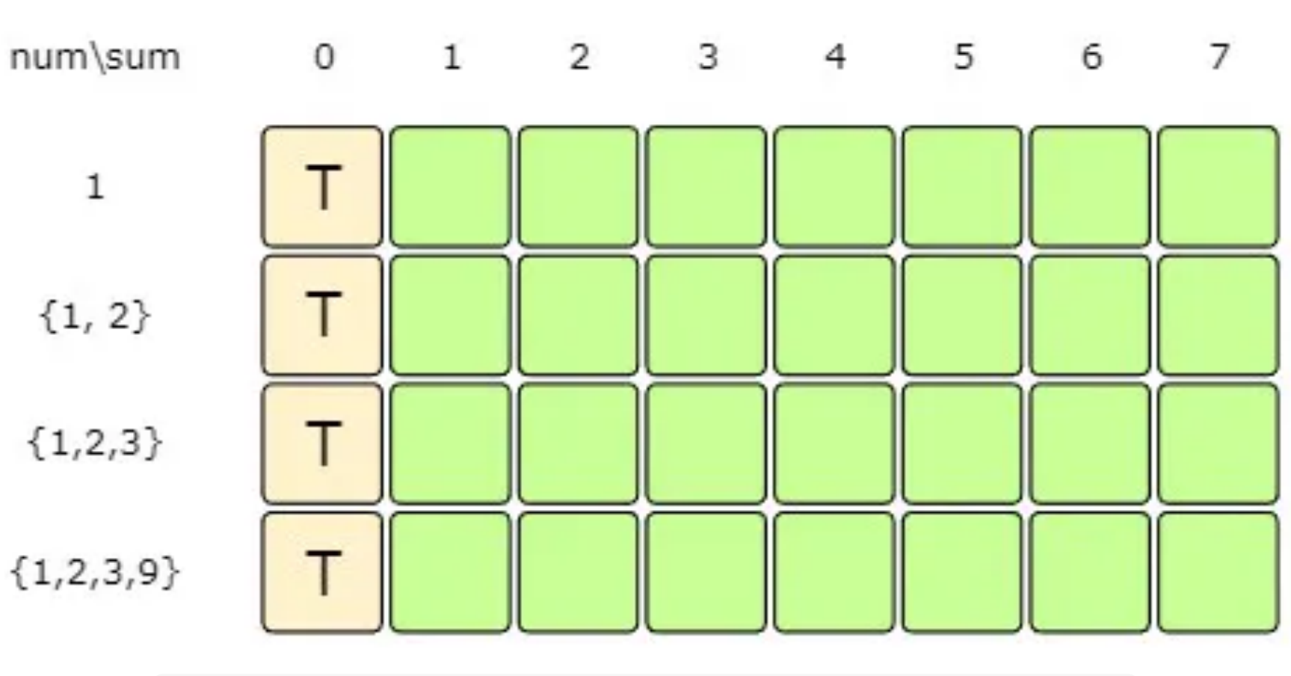
Essentially, we need to calculate all the possible sums up to 'S/2' for all numbers. So how do we populate the array `db[TotalNumbers][S/2+1]` in the bottom-up fashion?

For every possible sum ‘s’ (where 0 <= s <= S/2), we have two options:

1. Exclude the number. In this case, we will see if we can get the sum 's' from the subset excluding this number => `dp[index-1][s]`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum => `dp[index-1][s-num[index]]`

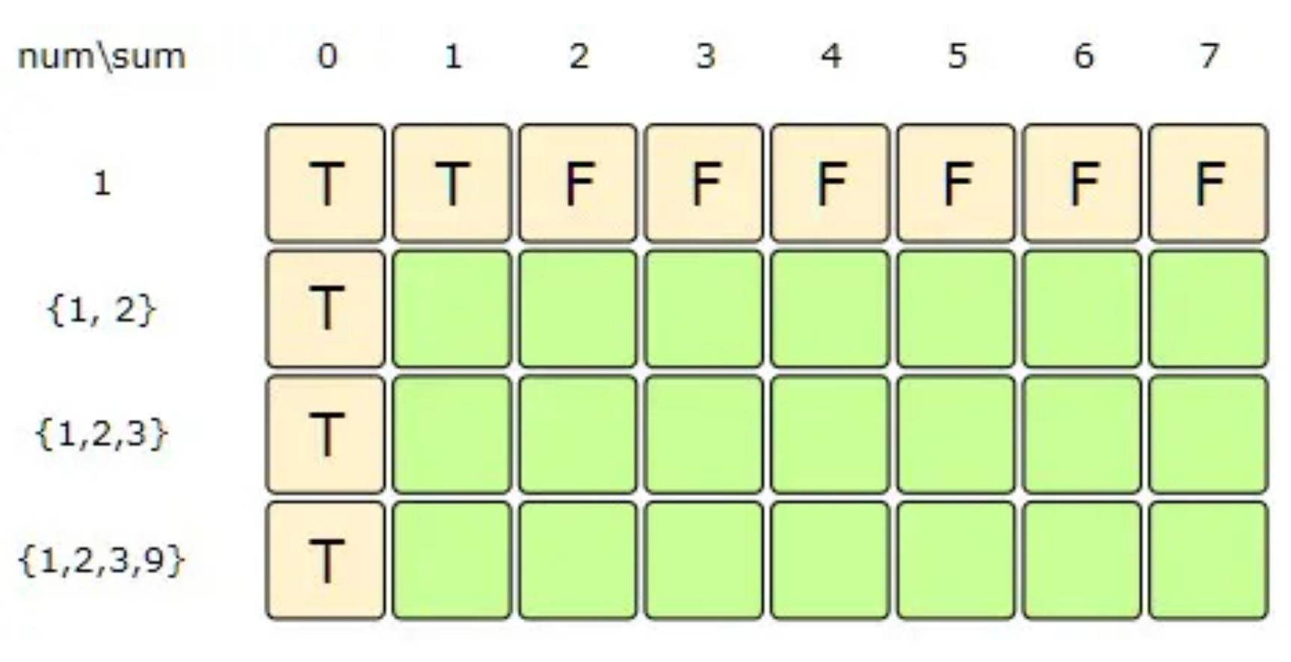
If either of the two above scenarios is true, we can find a subset with a sum equal to 's'. We should dig into this before we can learn how to find the closest subset.

Let's draw this visually, with the example input {1, 2, 3, 9}. Since the total sum is '15', therefore, we will try to find a subset whose sum is equal to the half of it i.e. '7'.



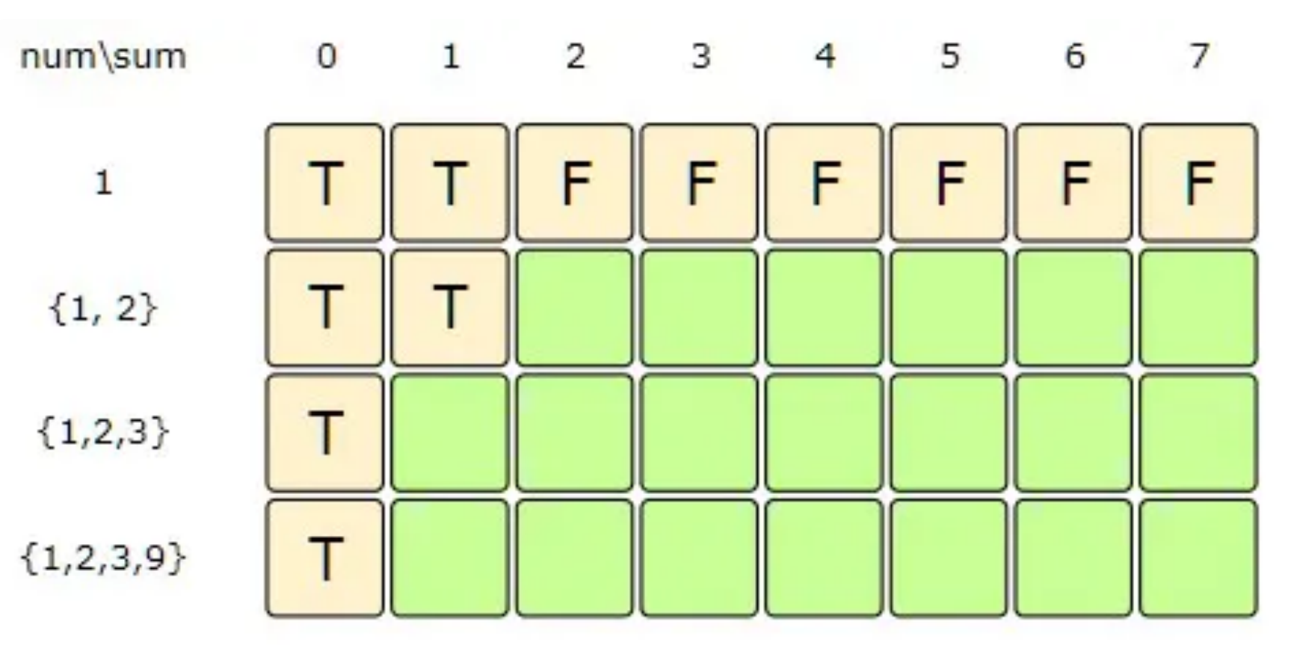
num\sum	0	1	2	3	4	5	6	7
1	T							
{1, 2}	T							
{1,2,3}	T							
{1,2,3,9}	T							

'0' sum can always be found through an empty set



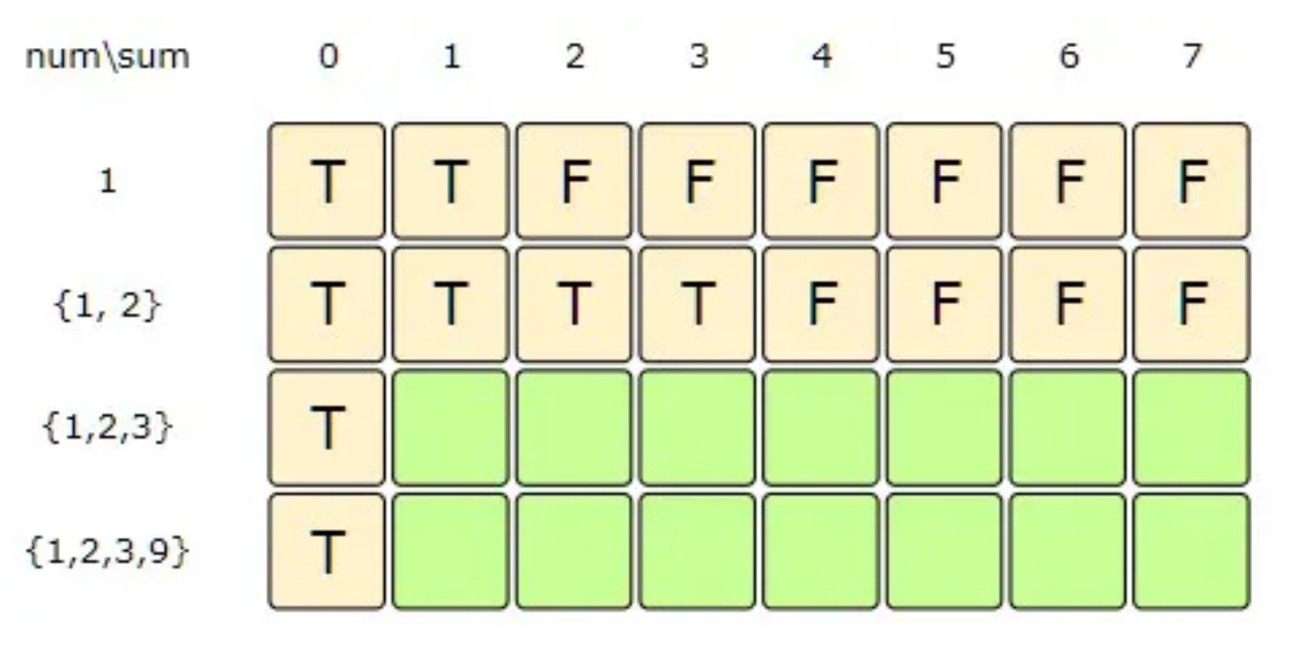
num\sum	0	1	2	3	4	5	6	7
1	T	T	F	F	F	F	F	F
{1, 2}	T							
{1,2,3}	T							
{1,2,3,9}	T							

With only one number, we can form a subset only when the required sum is equal to that number



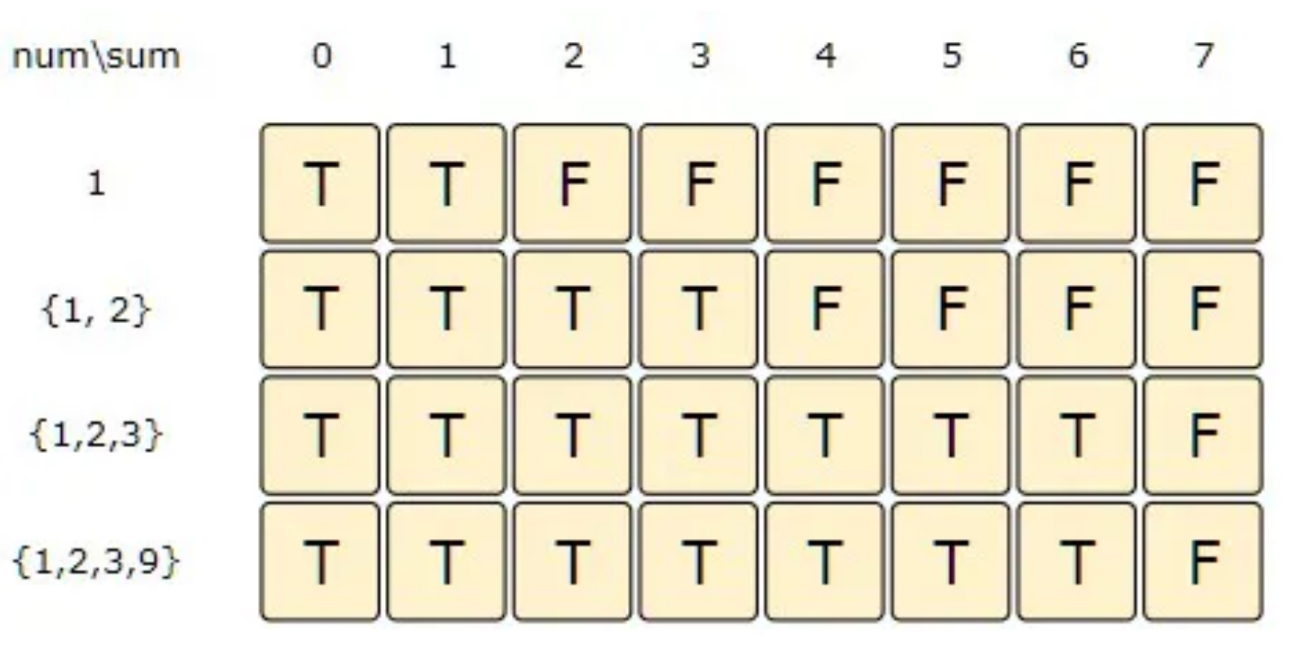
num\sum	0	1	2	3	4	5	6	7
1	T	T	F	F	F	F	F	F
{1, 2}	T	T						
{1,2,3}	T							
{1,2,3,9}	T							

sum: 1, index:1=> (dp[index-1][sum] , as the 'sum' is less than the number at index '1' (i.e., 1 < 2)



num\sum	0	1	2	3	4	5	6	7
1	T	T	F	F	F	F	F	F
{1, 2}	T	T	T	T	F	F	F	F
{1,2,3}	T							
{1,2,3,9}	T							

sum: 4-7, index:1=> (dp[index-1][sum] || dp[index-1][sum-2])



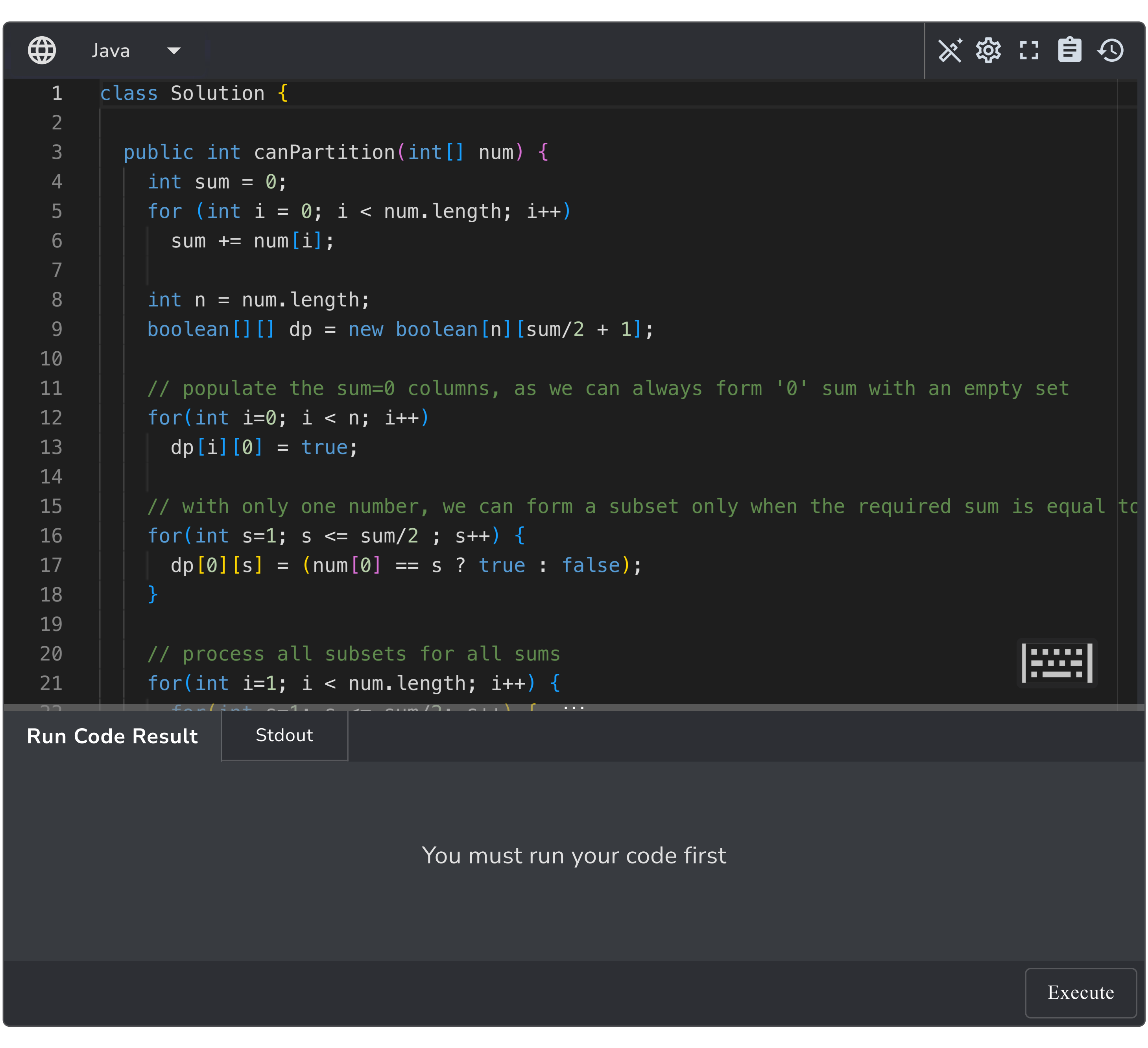
num\sum	0	1	2	3	4	5	6	7
1	T	T	F	F	F	F	F	F
{1, 2}	T	T	T	T	F	F	F	F
{1,2,3}	T	T	T	T	T	T	T	F
{1,2,3,9}	T	T	T	T	T	T	T	F

sum: 1-7, index:1=> (dp[index-1][sum] , as the 'sum' is always less than the number (9)

The above visualization tells us that it is not possible to find a subset whose sum is equal to '7'. So what is the closest subset we can find? We can find such a subset if we start moving backward in the last row from the bottom right corner to find the first 'T'. The first “T” in the above diagram is the sum ‘6’, which means we can find a subset whose sum is equal to '6'. This means the other set will have a sum of '9', and the minimum difference will be '3'.

Code

Here is the code for our bottom-up dynamic programming approach:



```java
class Solution {

  public int canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    int n = num.length;
    boolean[][] dp = new boolean[n][sum/2 + 1];

    // populate the sum=0 columns, as we can always form '0' sum with an empty set
    for(int i=0; i < n; i++)
      dp[i][0] = true;

    // with only one number, we can form a subset only when the required sum is equal to
    for(int s=1; s <= sum/2 ; s++) {
      dp[0][s] = (num[0] == s ? true : false);
    }

    // process all subsets for all sums
    for(int i=1; i < num.length; i++) {
```

You must run your code first

The above solution has time and space complexity of $O(N * S)$, where ‘N’ represents total numbers and ‘S’ is the total sum of all the numbers.

← Previous
Minimum Subset Sum Difference

Next →
Count Of Subset Sum

☐ Mark as Completed